

contraction-lean: Formal Proofs of Contraction Theory, Banach Fixed Point, and Exponential Convergence in Lean 4

Ben Cassie
2026

Abstract

contraction-lean is a Lean 4 / Mathlib library formalising core results from contraction theory: Banach fixed points, explicit geometric convergence rates, composition and iteration rules for contracting maps, derivative-based contraction criteria, and exponential trajectory convergence. The library defines `IsContracting` as a clean local API over Mathlib's `ContractingWith`, then reuses Mathlib's fixed-point and Lipschitz infrastructure while adding standalone theorem names suited to dynamical-systems work. Its headline results include existence and uniqueness of fixed points, convergence of iterates, the explicit rate $c^n/(1-c)$, composition of contraction constants, a mean-value-theorem bridge from derivative bounds to contractions, and a Gronwall-style exponential decay theorem. The development contains zero `sorry`, zero `admit`, and no project-specific axioms. Its significance is that contraction is stronger than Lyapunov stability: a contracting system does not merely remain stable or descend in a potential; it exponentially forgets initial conditions.

1. Introduction

Contraction theory studies maps and dynamical systems that bring points closer together. In its simplest metric form, a map $f : X \rightarrow X$ is a contraction when there is a constant $c < 1$ such that:

$$\text{dist}(f(x), f(y)) \leq c * \text{dist}(x, y)$$

for all points x, y . This single inequality has strong consequences. On a complete nonempty metric space, a contraction has a unique fixed point, every iterate converges to that fixed point, and the convergence has an explicit geometric rate. This is the Banach fixed-point theorem, one of the central tools of analysis.

In dynamical-systems form, contraction theory goes further. If distances between neighbouring trajectories shrink at a uniform differential rate, then any two trajectories converge toward each other exponentially. This is the perspective developed in modern nonlinear control, especially in the contraction analysis of Lohmiller and Slotine. A contracting system forgets its initial conditions: different starting states are pulled together by the dynamics.

This property is stronger than ordinary Lyapunov stability. A Lyapunov argument may show that an energy decreases, that trajectories remain bounded, or that a particular equilibrium is stable. Contraction gives a relational statement about pairs of trajectories. It says not only that one trajectory behaves well, but that all trajectories in the contracting region converge toward each other at a rate controlled by the contraction constant.

contraction-lean formalises a compact proof spine for this theory in Lean 4 / Mathlib. The library wraps Mathlib’s existing `ContractingWith` API in a local predicate `IsContracting`, proves Banach fixed-point consequences under convenient theorem names, provides composition and iteration rules for building contracting systems, proves a derivative criterion for real functions using a mean-value-theorem Lipschitz bound, and formalises a Gronwall-style exponential decay argument for trajectory convergence.

The contribution is not a new contraction theorem. It is a machine-checked, importable formalisation of standard contraction-theoretic mechanisms under precise hypotheses. This matters because contraction arguments are widely used in control theory, synchronisation, distributed systems, neural dynamics, optimisation, and AI safety. If future formal work is to reason about robust convergence or forgetting of initial conditions, it needs reusable checked primitives for contraction.

2. Library Overview

The project is organised into four Lean modules plus a root import file:

- `ContractionLean/Defs.lean` defines `IsContracting` and basic API wrapping Mathlib’s `ContractingWith`.
- `ContractionLean/BanachFixed.lean` proves the unique fixed point theorem, geometric convergence rate, and iterate convergence.
- `ContractionLean/CompositionRules.lean` proves composition and iteration rules for contracting maps.
- `ContractionLean/DifferentialContraction.lean` proves derivative-based contraction, Gronwall decay, and trajectory convergence.
- `ContractionLean.lean` is the root module importing the library.

The project depends on:

- Lean v4.28.0
- Mathlib v4.28.0

The metric setting is a complete metric or emetric space X , a contraction constant $c < 1$, and a self-map $f : X \rightarrow X$ satisfying:

$$\text{dist}(f(x), f(y)) \leq c * \text{dist}(x, y).$$

In the Lean source, the local predicate is:

```
def IsContracting {α : Type*} [EMetricSpace α] (c : ℝ≥0) (f : α → α) : Prop :=
  ContractingWith c f
```

Thus `IsContracting` is intentionally definitionally aligned with Mathlib’s `ContractingWith`. It gives this repository a domain-specific name while preserving access to Mathlib’s fixed-point and Lipschitz lemmas.

The differential setting has two parts. First, for $f : \mathbb{R} \rightarrow \mathbb{R}$, a pointwise derivative bound

$$\|f'(x)\| \leq c < 1$$

implies that f is a c -contracting map. Second, for trajectory distances, a scalar differential inequality

$$u'(t) \leq -\mu * u(t)$$

with $\mu > 0$ implies exponential decay:

$$u(t) \leq u(0) * \exp(-\mu t).$$

The repository is available at:

<https://github.com/velvetmonkey/contraction-lean>

3. Theorem Inventory

The library contains nine headline results, arranged in three layers: metric contraction, composition rules, and differential contraction.

Layer 1 - Metric Contraction

1. `IsContracting` — A map is contracting when it is a `ContractingWith` map, i.e. it has contraction constant $c < 1$ and is Lipschitz with constant c :

```
def IsContracting {α : Type*} [EMetricSpace α] (c : ℝ≥0) (f : α -> α) : Prop :=
  ContractingWith c f
```

The surrounding API exposes the strict constant bound, Lipschitz property, distance inequality, continuity, and conversions to and from Mathlib's `ContractingWith`.

2. `exists_unique_fixedPoint` — A contraction on a nonempty complete metric space has a unique fixed point:

```
theorem exists_unique_fixedPoint (hf : IsContracting c f) :
  ∃! x : α, f x = x
```

This packages Banach's fixed-point conclusion as an existential uniqueness statement.

3. `geometric_convergence` — Iterates satisfy an explicit a priori geometric convergence bound:

```
theorem geometric_convergence (hf : IsContracting c f) (x : α) (n : ℕ) :
  dist (f^[n] x) (fixedPoint hf) ≤
    (c : ℝ) ^ n / (1 - (c : ℝ)) * dist x (f x)
```

This is stronger than a bare convergence statement because it gives a concrete rate and constant.

4. `tendsto_iterate_fixedPoint` — Iterates converge to the fixed point:

```
theorem tendsto_iterate_fixedPoint (hf : IsContracting c f) (x : α) :
  Tendsto (fun n => f^[n] x) atTop (nhds (fixedPoint hf))
```

Together, the fixed-point and convergence theorems establish the metric contraction spine: unique equilibrium, convergence of iterates, and an explicit geometric bound.

Layer 2 - Composition Rules

5. `IsContracting.comp` — The composition of a c_1 -contracting map and a c_2 -contracting map is $(c_1 * c_2)$ -contracting:

```
theorem comp {c1 c2 : ℝ≥0} {f g : α -> α}
  (hf : IsContracting c1 f) (hg : IsContracting c2 g) :
  IsContracting (c1 * c2) (f ∘ g)
```

This supports modular construction of contracting systems.

6. `IsContracting.iterate` — The n -th iterate of a c -contracting map is c^n -contracting for $n > 0$:

```
theorem iterate {c : ℝ≥0} {f : α -> α}
  (hf : IsContracting c f) {n : ℕ} (hn : 0 < n) :
  IsContracting (c ^ n) (f^[n])
```

This theorem expresses the exponential strengthening of contraction under repeated application.

Layer 3 - Differential Contraction

7. `IsContracting.of_deriv_le` — A derivative bound implies contraction for real functions:

```
theorem IsContracting.of_deriv_le {c : ℝ≥0} (hc : c < 1) {f : ℝ -> ℝ}
  (hf_diff : Differentiable ℝ f)
  (hf_bound : ∀ x : ℝ, ||deriv f x|| ≤ c) :
  IsContracting c f
```

The theorem uses Mathlib's mean-value-theorem infrastructure to turn a pointwise derivative bound into a global Lipschitz bound.

8. `exponential_decay_of_deriv_le` — A scalar differential inequality implies exponential decay:

```
theorem exponential_decay_of_deriv_le
  {u : ℝ -> ℝ} {T mu : ℝ} (_hmu : 0 < mu) (hT : 0 ≤ T)
  (hu_cont : ContinuousOn u (Icc 0 T))
  (hu_diff : DifferentiableOn ℝ u (interior (Icc 0 T)))
  (_hu_nonneg : ∀ t ∈ Icc 0 T, 0 ≤ u t)
  (hu_deriv : ∀ t ∈ interior (Icc 0 T), deriv u t ≤ -mu * u t) :
  ∀ t ∈ Icc 0 T, u t ≤ u 0 * exp (-mu * t)
```

This is the Gronwall-style decay lemma used for trajectory convergence.

9. `trajectory_convergence` — If the distance between two trajectories satisfies the contraction differential inequality, then the trajectories converge exponentially:

```
theorem trajectory_convergence
  {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
  {x1 x2 : ℝ -> E} {T mu : ℝ} (hmu : 0 < mu) (hT : 0 ≤ T)
  (hx1 : ContinuousOn x1 (Icc 0 T))
```

```

(hx₂ : ContinuousOn x₂ (Icc 0 T))
(hdelta_diff : DifferentiableOn ℝ (fun t => ||x₁ t - x₂ t||) (interior (Icc 0 T)))
(hdelta_deriv : ∀ t ∈ interior (Icc 0 T),
  deriv (fun t => ||x₁ t - x₂ t||) t ≤ -μ * ||x₁ t - x₂ t||) :
∀ t ∈ Icc 0 T, ||x₁ t - x₂ t|| ≤ exp (-μ * t) * ||x₁ 0 - x₂ 0||

```

This is the formal trajectory-level exponential forgetting theorem.

4. Key Technical Highlights

Wrapping ContractingWith

The definition of `IsContracting` is intentionally simple:

```
IsContracting c f := ContractingWith c f
```

This design avoids duplicating Mathlib’s contraction infrastructure. Mathlib already knows that `ContractingWith` maps are Lipschitz, continuous, have fixed points in complete nonempty metric spaces, and have iterate convergence properties. The local wrapper gives this repository a domain-specific API while preserving those results.

The wrapper also makes the theorem statements read like contraction theory rather than like raw Mathlib plumbing. Results such as `exists_unique_fixedPoint`, `geometric_convergence`, and `trajectory_convergence` can be stated in the language of contracting maps, while proofs can delegate to Mathlib where the infrastructure already exists.

This is a useful pattern for formal libraries: create a thin local vocabulary over mature upstream abstractions, then add theorem names and glue lemmas suited to the domain.

Explicit Geometric Convergence

The theorem `geometric_convergence` gives more than existence of a fixed point and more than eventual convergence. It gives the quantitative a priori estimate:

$$\text{dist}(f^n(x), x^*) \leq c^n / (1-c) * \text{dist}(x, f(x)).$$

The factor c^n is the exponential decay term. The denominator $1-c$ arises from the geometric-series bound on the remaining tail of the contraction iterates. Intuitively, each step reduces distances by at least a factor of c , so the remaining distance to the fixed point is bounded by the sum of a geometric tail.

This explicit estimate is important because contraction theory is often used as a rate theorem. It does not merely say that a process converges; it says how fast the process forgets its initial error. The Lean theorem records that rate directly in the statement.

Gronwall via an Antitone Auxiliary

The theorem `exponential_decay_of_deriv_le` formalises the standard scalar argument behind continuous-time contraction. Given:

$$u'(t) \leq -\mu u(t)$$

define the auxiliary function:

$$v(t) = u(t) * \exp(\mu t).$$

Differentiating gives:

$$v'(t) = \exp(\mu t) * (u'(t) + \mu u(t)).$$

The assumed inequality implies $v'(t) \leq 0$, so v is antitone on the interval. Therefore $v(t) \leq v(0)$, and after dividing by $\exp(\mu t)$ one obtains:

$$u(t) \leq u(0) * \exp(-\mu t).$$

In Lean, the key monotonicity step uses `antitone0n_of_deriv_nonpos`. The proof has to track continuity on $[0, T]$, differentiability on the interval interior, positivity of the exponential, and the algebra converting $v(t) \leq v(0)$ into the final exponential bound.

Derivative Bound to Global Contraction

The theorem `IsContracting.of_deriv_le` is the bridge from differential information to metric contraction. For a differentiable real function $f : \mathbb{R} \rightarrow \mathbb{R}$, if:

$$\|f'(x)\| \leq c < 1$$

for every x , then the map is globally c -Lipschitz, hence contracting.

The proof uses Mathlib's mean-value-theorem based lemma:

`lipschitzWith_of_nnnorm_deriv_le`

This converts a pointwise derivative norm bound into a global Lipschitz bound. The strict inequality $c < 1$ then supplies the contraction side condition. The result is a compact formal version of a standard analysis principle: uniform derivative bounds control finite distances.

Standard Axioms Only

The library introduces no project-specific axioms. It is written against Lean 4 and Mathlib, uses ordinary classical mathematics where needed, and contains zero `sorry` and zero `admit`.

5. Relation to Sibling Libraries

`contraction-lean` is part of the same Lean 4 formalisation programme as `kuramoto-lean`, `gradient-descent-lean`, `hopfield-lean`, `nesterov-lean`, and `lotka-volterra-lean`.

`kuramoto-lean` formalises finite- N Kuramoto synchronisation dynamics, including Lyapunov descent and coupling contraction statements. `contraction-lean` extracts the contraction-theoretic spine into a standalone library: fixed points, contraction rates, composition, derivative criteria, and trajectory convergence.

`gradient-descent-lean` proves convergence through objective descent. Gradient descent approaches a minimiser because the objective decreases under the update.

contraction-lean proves a stronger relational property: trajectories or iterates move closer to each other, giving exponential forgetting rather than only descent of a scalar objective.

hopfield-lean formalises discrete Lyapunov descent for finite-state associative memory. Hopfield updates move downhill in an energy landscape until no non-trivial update remains. contraction-lean handles continuous and metric contraction, including trajectory-level exponential convergence.

nesterov-lean formalises accelerated optimisation through a carefully weighted Lyapunov potential. Its convergence is fast but tied to an optimisation objective and coefficient schedule. contraction-lean is the dynamical-systems complement: it studies systems whose flows or iterates contract distances directly.

lotka-volterra-lean formalises a conservative Hamiltonian system. Its Hamiltonian is conserved rather than decreased, and trajectories do not forget initial conditions. This makes it the natural opposite of contraction-lean: Lotka-Volterra preserves orbit structure, while contraction erases differences exponentially.

Together, these libraries cover several recurring mechanisms:

- scalar descent and optimisation convergence;
- accelerated Lyapunov decrease;
- finite-state energy descent;
- synchronisation and coupled dynamics;
- Hamiltonian conservation;
- contraction and exponential forgetting.

The shared value is that these mechanisms become precise, importable Lean artifacts.

6. Significance for AI Safety

Contraction is a strong stability property. In many safety-relevant settings, it is not enough to know that a single trajectory is bounded or that a scalar objective decreases. One wants to know that different initial states, perturbations, or implementation errors are forgotten by the dynamics. Contraction provides exactly that kind of guarantee.

This matters for AI safety because modern learning and control systems are dynamical. Training processes, recurrent networks, world-model updates, distributed agents, and feedback controllers all evolve over time. If a subsystem is contracting, then perturbations shrink at a quantifiable rate. If it is not contracting, then small differences may persist, amplify, or circulate.

The distinction between Lyapunov stability and contraction is important. A Lyapunov function can show that energy decreases along a trajectory, but it may not show that two trajectories converge to each other. Contraction is relational: it compares trajectories directly. This makes it a natural formal tool for robustness, synchronisation, observer design, and stable recurrent computation.

The theorem `trajectory_convergence` captures this idea in a reusable form. If the distance between two trajectories satisfies the contraction differential inequality, then the distance is bounded by:

$\exp(-\mu t) * \|x_1(0) - x_2(0)\|.$

That is an explicit exponential forgetting guarantee.

A machine-checked contraction theorem does not certify a deployed AI system by itself. It does, however, provide a reliable mathematical component for larger formal arguments. Future work can import these results to reason about stable observers, contracting recurrent networks, synchronising multi-agent systems, or robust optimisation dynamics.

The broader value is cumulative. `gradient-descent-lean` supplies objective descent, `nesterov-lean` supplies accelerated Lyapunov decrease, `hopfield-lean` supplies discrete attractor convergence, `lotka-volterra-lean` supplies conservative non-forgetting, and `contraction-lean` supplies exponential forgetting. Together they make the differences between these mechanisms explicit and machine-checkable.

7. Conclusion

`contraction-lean` provides a compact Lean 4 / Mathlib formalisation of core contraction theory. It defines a local `IsContracting` predicate over Mathlib’s `ContractingWith`, proves Banach fixed-point consequences, gives explicit geometric convergence bounds, establishes composition and iteration rules, formalises a derivative criterion for contraction, proves a Gronwall-style exponential decay lemma, and derives trajectory-level exponential convergence.

The project is deliberately focused. It does not formalise the full matrix-measure version of Lohmiller-Slotine contraction analysis, global ODE existence, stochastic contraction, or application-specific observer design. Instead, it supplies a reliable formal core for fixed points, contraction rates, derivative-to-Lipschitz reasoning, and exponential trajectory convergence. That core can now be imported and extended by future work on control theory, synchronisation, optimisation, neural dynamics, and AI safety.

References

Lohmiller, W., & Slotine, J.-J. E. (1998). *On contraction analysis for nonlinear systems*. *Automatica*, 34(6), 683-696.

Banach, S. (1922). *Sur les opérations dans les ensembles abstraits et leur application aux équations intégrales*. *Fundamenta Mathematicae*, 3, 133-181.

The Mathlib Community. (2024). *The Lean Mathematical Library*. GitHub repository. <https://github.com/leanprover-community/mathlib4>

Cassie, B. (2026). *kuramoto-lean*. Zenodo. DOI: 10.5281/zenodo.20468619. <https://doi.org/10.5281/zenodo.20468619>

Cassie, B. (2026). *gradient-descent-lean*. Zenodo. DOI: 10.5281/zenodo.20472996. <https://doi.org/10.5281/zenodo.20472996>

Cassie, B. (2026). *hopfield-lean*. Zenodo. DOI: 10.5281/zenodo.20474169. <https://doi.org/10.5281/zenodo.20474169>

Cassie, B. (2026). *nesterov-lean*. Zenodo. DOI: 10.5281/zenodo.20474481. <https://doi.org/10.5281/zenodo.20474481>

Cassie, B. (2026). *lotka-volterra-lean*. Zenodo. DOI: 10.5281/zenodo.20474669. <https://doi.org/10.5281/zenodo.20474669>

Cassie, B. (2026). *contraction-lean: Formal Proofs of Contraction Theory, Banach Fixed Point, and Exponential Convergence in Lean 4*. GitHub repository. <https://github.com/velvetmonkey/contraction-lean>